

Project Title:

Developing a Cloud-Based Application Using Microsoft Azure Services

Course:

Introduction to Cloud Computing CS 4037

Instructor:

Muhammad Sudais

Contents

1. Project Overview	3
2. Objectives.....	3
3. Project Requirements	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	4
4. Azure Services to be Used.....	4
5. Project Milestones.....	5
6. Deliverables	6
7. Evaluation Criteria.....	6
8. Sample Project Ideas.....	7
1. E-Commerce Web Application with Azure Functions	7
2. Serverless Inventory Management System	7
3. Restaurant Ordering System with Serverless Backend	7
4. Cloud-Based Learning Management System (LMS).....	8
5. Content Management System (CMS) for Blogs	8
6. Online Bookstore with Azure SQL Database and CDN.....	8
7. Fitness Tracking Application with Azure Machine Learning.....	9
8. Event Ticketing System.....	9
9. Travel Booking System with Azure Logic Apps and Event Grid	9
10. Healthcare Patient Management System	10
11. Serverless Video Sharing Platform	10
12. Personal Finance Tracker.....	10
13. Serverless Polling and Voting System	11
14. Real-Time Weather Forecasting Dashboard.....	11
15. Employee Timesheet Management System	11
9. References.....	12

1. Project Overview

This project focuses on designing, deploying, and managing a cloud-based web application using Microsoft Azure. The objective is to familiarize students with core Azure services, demonstrate cloud architecture design, and apply practical knowledge in a real-world scenario. Students will design an end-to-end application using various Azure services, including compute, storage, database, and networking services.

2. Objectives

The primary objectives of the project are:

1. To understand and utilize Microsoft Azure cloud services in a project.
2. To design a scalable and resilient application.
3. To gain hands-on experience in deploying and managing applications in Azure.
4. To learn best practices for cloud architecture, security, and cost optimization.
5. To implement Continuous Integration/Continuous Deployment (CI/CD) pipelines using Azure DevOps.

3. Project Requirements

3.1 Functional Requirements

- **Application:**
Develop a basic application using any technology stack (ASP.NET, Python, etc.). The application should have user authentication and role-based access control.
- **User Features:**
 - Users should be able to register and log in.
 - A simple CRUD functionality (e.g., managing user profiles or records).
- **API Integration or Azure Function App:**
 - API Integration
 - Expose an API for application data.
 - Ensure secure API access using Azure API Management.
 - Azure Function App
 - Create an Azure Function which adds value to your application's functionality.
 - Integrate this application with a function app for some kind of utility, time triggered or event driven activities

- **Cloud Hosting:**
 - Host the application using Azure App Service.
- **Database:**
 - Use Azure SQL Database or Cosmos DB to store application data.
- **Storage:**
 - Integrate Azure Blob Storage for storing user-uploaded files (images, documents, etc.).
- **Monitoring and Logging:**
 - Implement monitoring for the application using Azure Monitor or Azure Application Insights.
- **Security:**
 - Use Azure Key Vault to manage sensitive data (API keys, connection strings).
 - Implement Azure Active Directory (Azure AD) for user authentication.
- **Networking:** (Optional)
 - Set up Virtual Networks (VNETs) and use Network Security Groups (NSGs) for secure communication.

3.2 Non-Functional Requirements

- **Performance:**
The application should handle at least 100 concurrent users.
- **Scalability:**
The application should be able to scale up/down automatically based on user load.
- **Availability:**
The application should have a high uptime, targeting 99.95% availability.
- **Security:**
The application should follow security best practices, including encryption, identity, and access management.
- **Cost Optimization:**
Demonstrate cost management techniques (e.g., optimizing resources, using reserved instances).

4. Azure Services to be Used

- **Compute:**
Azure App Service, Azure Function Apps.
- **Storage:**
Azure Blob Storage, Azure File Storage (optional).

- **Databases:**
Azure SQL Database or Cosmos DB(Preferred).
- **Networking:** (Optional)
Azure Virtual Network (VNet), Azure Traffic Manager.
- **Security:**
Azure Active Directory, Azure Key Vault.
- **Monitoring & Diagnostics:**
Azure Monitor, Application Insights.
- **DevOps:** (Optional)
Azure DevOps for CI/CD, including pipelines, repositories, and deployment slots.

5. Project Milestones

1. **Milestone 1: Project Proposal (Week 2)**
 - Submit a proposal detailing the chosen application, architecture, and Azure services. It should also contain the details of the 5 group members and the project lead.
2. **Milestone 2: Application Design and Architecture (Week 4)**
 - Complete the design of the application's architecture on Azure.
 - Submit an architecture diagram showing how different Azure services will interact.
3. **Milestone 3: Initial Deployment (Week 8)**
 - Set up and deploy the application with basic features to Azure.
 - Implement user authentication and database integration.
4. **Milestone 4: Full Functionality & Monitoring (Week 12)**
 - Complete the application with all functional requirements.
 - Implement monitoring, logging, and performance optimization.
5. **Milestone 5: Final Project Submission (Week 16)**
 - Submit the full project with all required features and documentation.
 - Give a presentation demonstrating the project, its architecture, and Azure implementation.

6. Deliverables

- **Proposal Document:**
Initial project idea, architecture design, and services to be used.
- **Source Code:**
Full codebase hosted on GitHub or Azure DevOps, with proper documentation.
- **Azure Resources:**
Details of all Azure services used, including configurations, security policies, and cost estimates.
- **Final Report:**
A comprehensive report detailing the architecture, services, implementation challenges, and key learnings.
- **Presentation:**
A 10-minute presentation showcasing the application, cloud architecture, and deployment strategy.

7. Evaluation Criteria

S.No	Project Items	Marks
1	Architecture Design	15
2	Functionality	30
2.1	• Database & Storage (Code & Configuration)	15
2.2	• API or Function App	10
2.3	• Security (Key Vault and other functional security)	5
3	Cloud Implementation	30
3.1	• App Service Implementation	10
3.2	• API or Function App	5
3.3	• Security (AD Authentication etc.)	5
3.4	• Azure Monitoring	10
4	Documentation & Presentation	25
4.1	• Proposal	5
4.2	• Report	10
4.3	• Presentation & Demonstration	10

1. **Architecture Design (15%):**
Well-structured design using appropriate Azure services, including diagrams and explanations.
2. **Functionality (30%):**
Correct implementation of user features, API, Function App, database, storage, and security.

3. **Cloud Implementation (30%):**
Efficient use of Azure services (compute, storage, security, monitoring).
4. **Documentation & Presentation (25%):**
Clear, concise documentation and a professional project presentation.

8. Sample Project Ideas

1. E-Commerce Web Application with Azure Functions

Description:

Develop a cloud-based e-commerce platform where users can browse products, add items to their cart, and make purchases. Use Azure Functions for backend logic, and Azure Cosmos DB to store product information and user data.

Azure Services:

- Azure Functions
- Azure Cosmos DB
- Azure API Management
- Azure App Service
- Azure Blob Storage (for product images)

2. Serverless Inventory Management System

Description:

Develop a serverless inventory management system for a retail business using Azure Functions for backend operations and Azure Cosmos DB to store product and inventory data. Azure Blob Storage will be used for product images, while Azure API Management will expose APIs to external applications.

Azure Services:

- Azure Functions (for CRUD operations on inventory)
- Azure Cosmos DB (for storing product and inventory data)
- Azure API Management (for external APIs)
- Azure App Service (for hosting admin dashboard)
- Azure Blob Storage (for product images)

3. Restaurant Ordering System with Serverless Backend

Description:

Create a restaurant ordering system where customers can browse menus, place orders, and track their order status in real time. Azure Functions will handle order processing, Cosmos DB will store menu items, and Blob Storage will store images of the food.

Azure Services:

- Azure Functions (to manage order placement and updates)
- Azure Cosmos DB (for storing menu items and order data)
- Azure API Management (to expose APIs to partner apps)
- Azure App Service (for customer web app)
- Azure Blob Storage (for storing food images)

4. Cloud-Based Learning Management System (LMS)

Description:

Design a cloud-based LMS that allows teachers to upload course materials, and students to access content, submit assignments, and track their progress. Implement role-based access and use Azure services to ensure scalability.

Azure Services:

- Azure App Service
- Azure Cosmos DB (for storing user data and course materials)
- Azure Blob Storage (for document uploads)
- Azure Active Directory (for user roles)
- Azure DevOps (for CI/CD pipeline)

5. Content Management System (CMS) for Blogs

Description:

Build a cloud-based CMS where users can write, edit, and publish blog posts. Azure Blob Storage will be used for media uploads, Azure Cosmos DB will store blog content, and Azure Functions will handle automated tasks like post scheduling and notifications.

Azure Services:

- Azure Functions (for content scheduling and notifications)
- Azure Cosmos DB (to store blog posts and metadata)
- Azure API Management (to expose blog APIs)
- Azure App Service (for hosting the CMS)
- Azure Blob Storage (for media assets like images)

6. Online Bookstore with Azure SQL Database and CDN

Description:

Build an online bookstore where users can browse, search, and purchase books. Implement Azure Content Delivery Network (CDN) to optimize content delivery and Azure SQL Database for book and user information.

Azure Services:

- Azure App Service
- Azure SQL Database (for storing book and user data)
- Azure CDN (for optimizing book images and other assets)
- Azure Blob Storage (for storing book previews)
- Azure Functions (for handling search and user interactions)

7. Fitness Tracking Application with Azure Machine Learning

Description:

Develop a fitness tracking app where users can log workouts, track progress, and receive personalized fitness recommendations. Use Azure Machine Learning to analyze user data and provide custom workout plans.

Azure Services:

- Azure Machine Learning
- Azure App Service (for hosting the app)
- Azure Cosmos DB (for user and workout data)
- Azure Blob Storage (for storing user media like progress photos)
- Azure DevOps (for CI/CD pipeline)

8. Event Ticketing System

Description:

Design an event ticketing system where users can purchase tickets for concerts and other events. Use Azure Functions for payment processing and ticket validation, Azure Cosmos DB for storing event details, and Blob Storage for storing promotional materials.

Azure Services:

- Azure Functions (for payment processing and ticket validation)
- Azure Cosmos DB (for storing event and user data)
- Azure API Management (for external API integration)
- Azure App Service (for customer portal)
- Azure Blob Storage (for storing promotional images)

9. Travel Booking System with Azure Logic Apps and Event Grid

Description:

Design a travel booking system where users can search and book flights and hotels. Use Azure Logic Apps to automate booking confirmations and Azure Event Grid to handle real-time updates such as flight delays or cancellations.

Azure Services:

- Azure Function Apps
- Azure Event Grid
- Azure Cosmos DB (for storing booking data)
- Azure App Service
- Azure Cognitive Search (for travel search functionality) or some other service.

10. Healthcare Patient Management System

Description:

Build a healthcare patient management system that allows doctors to manage patient records, schedule appointments, and view medical history.

Azure Services:

- Azure Function Apps
- Azure Cosmos DB (for patient records)
- Azure Active Directory (for secure access control)
- Azure Blob Storage (for medical document storage)
- Azure Monitor (for application health monitoring)

11. Serverless Video Sharing Platform

Description:

Create a video-sharing platform where users can upload, watch, and share videos. Use Azure Blob Storage for video uploads, Azure Cosmos DB for user and video metadata, and Azure Functions to manage video processing (e.g., resizing, transcoding).

Azure Services:

- Azure Functions (for video processing and metadata updates)
- Azure Cosmos DB (to store user profiles and video metadata)
- Azure API Management (for video API exposure)
- Azure App Service (for the video-sharing frontend)
- Azure Blob Storage (for video storage)

12. Personal Finance Tracker

Description:

Develop a personal finance tracking app that lets users manage expenses, set budgets, and visualize spending habits. Use Azure Functions to process transaction data, Cosmos DB to store user accounts and transaction histories, and Blob Storage for user-uploaded receipts.

Azure Services:

- Azure Functions (to process and categorize transactions)
- Azure Cosmos DB (to store transaction history)
- Azure API Management (to expose finance-related APIs)
- Azure App Service (for a mobile/web finance dashboard)
- Azure Blob Storage (for storing receipts and documents)

13. Serverless Polling and Voting System

Description:

Build a serverless polling system where users can create polls, vote, and view results in real time. Azure Functions will handle voting logic, Azure Cosmos DB will store polls and votes, and Azure Blob Storage will handle any media (e.g., images, videos) linked to polls.

Azure Services:

- Azure Functions (for poll creation and vote handling)
- Azure Cosmos DB (for storing polls and results)
- Azure API Management (for exposing poll APIs)
- Azure App Service (for user and admin portals)
- Azure Blob Storage (for storing media attached to polls)

14. Real-Time Weather Forecasting Dashboard

Description:

Create a weather forecasting dashboard that pulls data from external APIs and displays real-time weather information. Use Azure Functions to gather weather data, Cosmos DB to store historical data, and Blob Storage for storing graphical weather reports.

Azure Services:

- Azure Functions (for pulling and processing weather data)
- Azure Cosmos DB (to store historical weather data)
- Azure API Management (to expose weather data APIs)
- Azure App Service (for the weather dashboard frontend)
- Azure Blob Storage (for storing weather reports and images)

15. Employee Timesheet Management System

Description:

Design a timesheet management system where employees can log their work hours and managers can approve or reject entries. Use Azure Functions to process timesheet submissions, Cosmos DB to store employee data, and Blob Storage for storing scanned timesheets.

Azure Services:

- Azure Functions (for timesheet submission and approval logic)
- Azure Cosmos DB (to store employee and timesheet data)
- Azure API Management (for external app integration)
- Azure App Service (for employee and admin dashboards)
- Azure Blob Storage (for storing scanned documents)

9. References

- Microsoft Azure Documentation: <https://docs.microsoft.com/en-us/azure/>
- Azure Architecture Guide: <https://docs.microsoft.com/en-us/azure/architecture/>
- Azure DevOps: <https://azure.microsoft.com/en-us/services/devops/>